

Performance Analysis and Walltime Prediction for Neuroscience Applications

Axel Vivenot
axel.vivenot@telecom-paris.fr

Valentin Delis
valentin.delis@ensiie.fr

Abstract—With the recent upcomming of HPC, BigData and AI, the need for computational resources is ever increasing in the field of neuroscience. However, efficiently utilizing these resources remains a challenge due to the complexity of modern computing systems and the diverse nature of neuroscience applications. In this paper, we present a comprehensive performance analysis of several neuroscience applications, focusing on their execution time and memory footprint. In a second step, we will compare these results with other previous experiments done on some of these algorithms, then we will develop a model to predict the execution time and memory usage of these applications based on their input parameters and system configurations.

Index Terms—Stochastic application, Execution time, Memory footprint

I. INTRODUCTION

When designing an application, it is crucial to know whether it will run consistently depending on the input parameters. This is especially true for stochastic applications, where the execution time can vary significantly based on the input. Moreover, in the context of High-Performance Computing (HPC), where resources are often allocated based on estimated execution times, having accurate predictions can lead to better resource management and cost savings. With the increasing complexity of neuroscience applications due to the rise of AI and Big Data, it becomes even more important to understand and predict their performance characteristics. In this paper, we focus on analyzing the performance of several neuroscience applications, with the goal of predicting their execution time and memory footprint based on input parameters.

II. STUDY OF NEUROSCIENCE APPLICATIONS

In this section, we will study multiple neuroscience applications with different datasets and input parameters. For each application, we will measure the execution time and memory footprint for various input configurations.

A. Platform

All experiments were conducted on a machine with the following specifications:

- CPUs: 2 AMD EPYC 7502 32-Core Processor (64 threads @ 2.50GHz)
- RAM: 504 GiB
- GPU: NVIDIA Quadro RTX 5000

When running the applications, we ensured that no other significant processes were running on the machine to avoid interference with the measurements. Still, some background

processes were taking on average 1% of CPU and 6% of RAM (i.e. around 30GiB of RAM).

B. Dataset

We used the Dartmouth Raiders Dataset (DRD) for our experiments. We extracted two separate datasets from the DRD:

- A dataset containing 87 images of fMRI (Functional magnetic resonance imaging) scans. Each is a 4D scan, with 326 pictures at resolution of 80x80x41 voxels.
- A dataset containing 21 images of T1-weighted MRI (Magnetic resonance imaging) scans. Each is a 3D scan, with a resolution of 256x256x160 voxels.

C. Spatially Localized Atlas Network Tiles (SLANT)

We study the SLANT algorithm, which is a deep learning-based method for whole brain segmentation. Although SLANT is supposed to take 3D, T1-weighted MRI scans as input, we tested it with fMRI scans to compare with previous experiments done on SLANT which used the same fMRI dataset. We studied both the CPU and GPU implementations of SLANT, taking into account both the execution time and memory footprint of SLANT with two different runtimes:

- Docker
- Singularity
- a) *SLANT CPU Singularity:*

In this part, we study how the SLANT CPU implementation with Singularity performs depending on the number of threads used. We tested the algorithm with 1 thread, 48 threads and 128 threads.

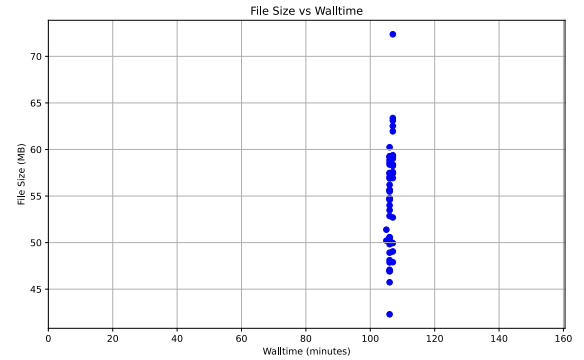


Fig. 1. SLANT CPU Singularity Walltime vs fMRI number of scans for one threads

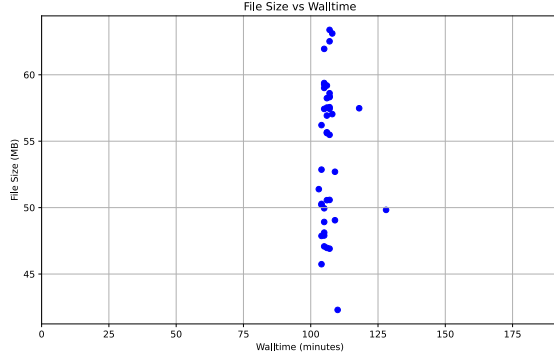


Fig. 2. SLANT CPU Singularity Walltime vs fMRI number of scans for 48 threads

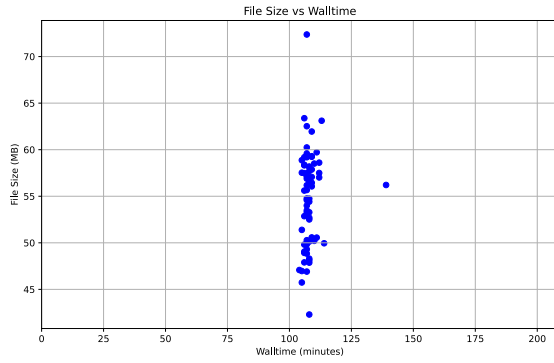


Fig. 3. SLANT CPU Singularity Walltime vs fMRI number of scans for all 128 threads

We can observe in Figure 1, Figure 2 and Figure 3 that the walltime does not vary significantly with the number of threads used. Moreover, the walltime does not seem to vary depending on the input, staying at around 110 minutes. The times are displayed on the table below:

TABLE I
MEAN AND STANDARD DEVIATION OF SLANT CPU SINGULARITY WALLTIME (IN SECONDS)

Nb of threads	Mean time (min)	Std Dev (min)
1	106.33	0.55
48	106.79	4.12
128	108.12	4.11

TABLE II
MEAN AND STANDARD DEVIATION WALLTIME OF EACH STEP FOR SLANT CPU SINGULARITY (IN SECONDS)

Nb of threads	Preprocessing (mean)	Preprocessing (std)	Segmentation (mean)	Segmentation (std)	Post-processing (mean)	Post-processing (std)
1	931	25	4820	11	588	8
48	836	39	4906	212	660	55
128	856	41	4924	78	705	224

We see that, in each step of the algorithm, the standard deviation is very low compared to the mean time, showing that the execution time is very consistent regardless of the input.

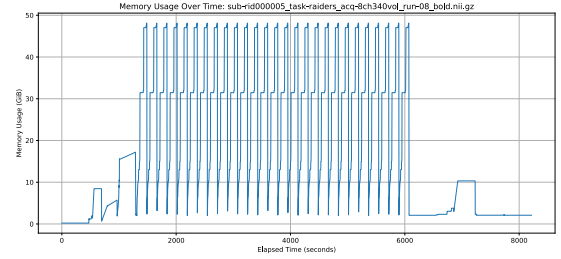


Fig. 4. SLANT CPU Docker memory profile for all 128 threads

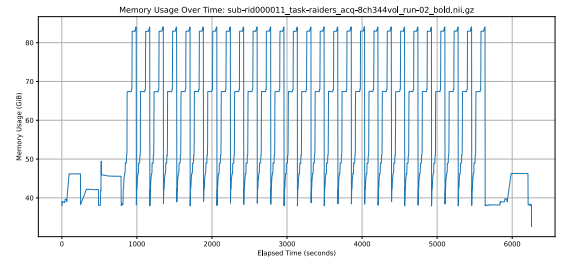


Fig. 5. SLANT CPU Singularity memory profile for all 128 threads. Notice the Y-axis does not start at 0 due to the high background memory usage.

TABLE III
MEAN AND STANDARD DEVIATION WALLTIME OF DEEPMRIPREP IN SECONDS

Type	Nb of threads	Output type	Walltime (mean)	Walltime (std)
cpu	1	all	326.5	2.3
cpu	128	all	85.6	1.6
cpu	128	rbm	51.1	1.0
gpu	128	vbm	17.0	1.0

REFERENCES